

Scalable Hardware Pruning through Semiformal Verification and Microarchitecture Awareness

ABSTRACT

As we approach the practical limits of Moore’s Law and move beyond Dennard scaling, Application Specific Integrated Processor (ASIP) has attracted increasing interest due to its potential for trading generality for reduced area, improved performance, and lower power consumption. Efficient and effective pruning of unused hardware under application-specific constraints constitutes a fundamental challenge in ASIP design. While existing approaches have made significant progress, they remain fundamentally limited by attempting complete state space exploration while maintaining microarchitecture independence—the former causing severe scalability bottlenecks and the latter sacrificing substantial optimization opportunities. In this work, we propose an agile versatile hardware pruning framework for ASIP design that embraces both semiformal analysis and microarchitecture awareness. First, we employ bounded model checking and k-step induction, trading proof completeness for practical scalability—enabling analysis of industrial-scale designs that would be intractable under unbounded verification. Second, we translate microarchitecture-aware constraints—including ISA, data, and timing-level constraints—into External Don’t Care (EDC) and Observability Don’t Care (ODC), exposing optimization opportunities invisible to architecture-agnostic methods. Third, we define a Constraint-based RISC-V Instruction Specification Protocol (CRISP) using denotational semantics to facilitate the expression and SystemVerilog assertion (SVA) generation of all aforementioned constraints.

Under identical constraints, our framework produces implementations that are up to 49.6% smaller than state-of-the-art additive ASIP design methodology (RISSP). Relative to prior hardware-trimming frameworks, it achieves 3–4 orders of magnitude faster analysis runtime and delivers 1.96× greater area reduction on the same constrained multiplier benchmark.

KEYWORDS

Formal method, ASIP, Observability Don’t Care

1 INTRODUCTION

Resource-constrained computing platforms continue to expand across diverse application domains, from IoT edge devices to implantable biomedical systems to flexible electronics for wearable computing. These systems face stringent constraints on silicon area, power budgets, and manufacturing costs that make conventional processor architectures increasingly impractical. Flexible and printed electronics exemplify this challenge: with transistor counts measured in thousands rather than billions and mask costs approaching zero, the economics fundamentally favor highly specialized designs over general-purpose solutions [2, 11]. ASIPs offer a promising approach to these challenges by customizing processor architectures to specific workload requirements, enabling dramatic reductions in area and power consumption compared to general-purpose designs.

Traditional ASIP methodologies primarily focus on enhancing performance through instruction set extensions, adding specialized operations to accelerate critical computations. However, this additive strategy contradicts the fundamental need for smaller, more efficient processors in resource-constrained environments. Adding custom instructions increases rather than decreases silicon footprint, exacerbating the very problem these systems need to solve. Recent analysis of production workloads reveals a compelling alternative: typical applications exercise only a fraction of available processor functionality [3, 13], with large portions of hardware remaining dormant throughout execution. This observation motivates a subtractive approach to specialization, eliminating unused capabilities rather than adding new ones.

Recognition of this opportunity has spurred significant research into hardware pruning techniques for processor optimization. Pioneering work on Bespoke microprocessors [5] established the feasibility of automatically tailoring processors to individual applications by removing untoggled gates. Subsequent efforts expanded this concept: PDAG [3] introduced formal verification methods to ensure correctness while supporting diverse ISAs and microarchitectures, and FSYN [9] provided open-source implementations of these techniques; RISC-V Instruction Subset Processors (RISSP) [13] focused specifically on RISC-V implementations with simplified microarchitectures. Additional research directions include approximate behavioral processors [16] that exploit error tolerance for further optimization, enhanced symbolic execution methods [15] for improved analysis coverage, accelerator subsetting [18] that extends these concepts beyond processors, and formal profiling techniques [17] for systematic workload characterization.

However, these existing approaches share fundamental limitations that constrain their practical applicability and optimization potential. First, subtractive processes are inherently conservative: they must formally prove that hardware structures are unnecessary, requiring expensive symbolic execution or model checking that often becomes intractable for industrial-scale designs. RISSP addresses this scalability challenge by adopting a hardware-additive approach—starting from a base architecture with no functional units and adding them as necessary—thereby avoiding the need for expensive proofs of unnecessary [13]. Second, most approaches remain microarchitecture-unaware, considering only EDCs at the ISA level. This architectural myopia prevents them from exploiting data-level and interface-level constraints, and critically, causes them to overlook substantial ODCs that emerge from the interaction between ISA-level controllability constraints and microarchitectural implementation details. Third, while RISSP achieves both scalability and microarchitecture awareness, its hardware-additive approach requires committing to a specific microarchitectural template, sacrificing the generality required for diverse design contexts—a limitation analogous to ISA-additive approaches. The open-source FSYN is only demonstrated on processor subcircuits (e.g., controller, multiplier), and scales poorly up to full microprocessors.

What is needed is a hardware pruning framework that addresses all three limitations simultaneously. Such a tool must satisfy four critical requirements: (1) open-source availability to minimize non-recurring engineering costs—particularly important in domains like flexible and printed electronics where EDA licensing costs can exceed mask and equipment costs, (2) scalability to and compatibility with arbitrary microcontroller microarchitecture implementations rather than fixed templates, (3) support for expressing non-ISA constraints including data-level and interface-level specifications, and (4) microarchitecture-aware optimization capabilities that enable targeted exploitation of ODCs with acceptable computational overhead.

In this work, we propose Property-Driven Analysis and Generation via Semiformal Analysis (PDAG-SFA), a hardware pruning framework for ASIP design that combines property-driven analysis with signal correspondence to achieve both semiformal verification and microarchitecture awareness. Our approach makes three key contributions. First, we employ semiformal techniques to automated hardware removal, which enables practical scalability and speed, trading proof completeness for the ability to analyze industrial-scale designs that would be intractable or time consuming under unbounded verification. Second, we translate microarchitecture-aware constraints—including ISA-level, data-level, and interface-level specifications—into EDCs and ODCs, exposing optimization opportunities that remain invisible to microarchitecture-agnostic methods. Third, we define a domain-specific language (CRISP) using denotational semantics to facilitate precise expression and integration of all aforementioned constraints.

Through these contributions, PDAG-SFA delivers up to 49.6% area reduction compared with the state-of-the-art additive ASIP design approach. Relative to prior trimming frameworks, it achieves 3–4 orders of magnitude faster analysis and provides up to 1.96× greater area reduction on the same benchmark under the same constraints.

2 EFFICIENTLY EXPLOITING EDCs FOR LOGIC OPTIMIZATION

Complex EDCs arising from program behavior are difficult to exploit in logic synthesis. Bespoke Microprocessors [5] uses symbolic execution of a single program on a gate-level model to identify gates with invariant outputs (constant ‘0’ or ‘1’ at all times). These gates can then be eliminated from the design, and their input cones optimized. The resulting netlist can correctly execute the single program, but any other workload may fail to execute correctly. PDAG [3] addressed this generalizability issue, optimizing netlists for arbitrary program subsets expressible as Linear Temporal Logic (LTL) formulae. Rather than symbolic execution, PDAG uses LTL model checking to identify invariant gate outputs.

Both of these approaches face severe computational challenges. Symbolic execution fails for many programs, as code reachability is undecidable in general. Further, even when reachability is decidable, the path explosion problem—the exponential growth of execution paths at every data-dependent branch—leads to prohibitively long execution times. LTL model checking is PSPACE-complete in the size of the LTL formula and has runtime exponential in the number of flip-flops in the model. These computational complexities

result in days-to-months of CPU time even for small cores (Table 2) [9], and LTL model checking additionally requires expensive commercial multi-threaded solver licenses.

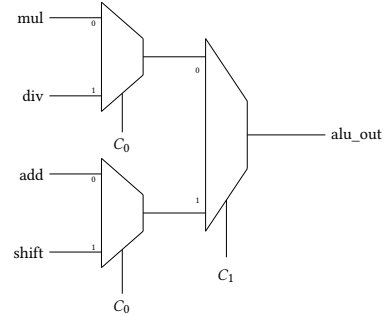


Figure 1: Prior works’ EDC analysis performed at gate-level granularity may miss controllability don’t cares which span across multiple gates, such as $C_0C_1 \neq 2b'10$.

These prior approaches, focused on individual gate outputs, fail to capture conditional dependencies between signals. Consider Fig. 1. When analyzing the circuit at gate-by-gate granularity, the EDC $C_0C_1 \neq 2b'10$ cannot lead to elimination of the *div* input, since even under this constraint, each multiplexer can still select each of its **direct** inputs. Error injection can identify such optimizations—by injecting an error into the *div* input and proving sequential equivalence between the error-injected and original circuits, we can safely optimize away the *div* input [7]. However, this approach faces two critical challenges. First, the number of possible error injection sites grows linearly with circuit size, making this approach computationally impractical even for small microcontrollers. Second, only *architecturally visible state* (e.g., register files, CSR registers, flags) truly needs verification, yet naive analysis checks all state, missing optimization opportunities that arise from reasoning about architectural observability.

As a result of these high computational costs and conservative gate-level analysis limitations, designers may reasonably conclude that the marginal benefits do not justify the increased development cost and verification effort.

2.1 Property Driven Generation of Reduced ISA Hardware via Semiformal Analysis

PDAG-SFA addresses both of these fundamental issues through semiformal verification with microarchitectural awareness, as depicted in Fig. 2. The framework takes three inputs that together enable scalable, microarchitecture-aware optimization. First, the **RTL design** provides the hardware description to be optimized. Second, **CRISP constraints** (detailed in Section 3) express ISA-level constraints using our domain-specific language—these constraints specify which instructions the optimized design must support. Third, a **microarchitecture configuration file** provides structural information about the target design: signal naming conventions (e.g., clock, reset, instruction signals), module hierarchy and injection points for constraint insertion, pipeline depth for *k*-induction tuning, the structure of result multiplexers that select

between functional unit outputs and microarchitecturally specific data/timing level constraints for additional area reduction. This configuration enables PDAG-SFA to reason about which functional units become unobservable under given ISA constraints, guiding targeted optimization.

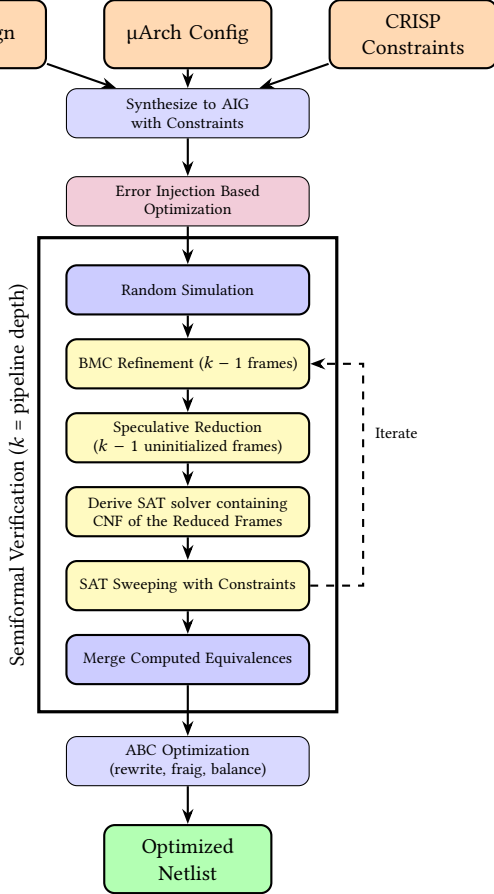


Figure 2: PDAG-SFA workflow: RTL design and CRISP constraints are synthesized to an AIG with embedded assumptions. Microarchitecture-aware error injection identifies candidate optimization targets, which are verified through semiformal SAT-based equivalence checking using k -induction at pipeline depth. Iterative refinement via random simulation, BMC, and speculative reduction enables scalable verification. Proven equivalences are merged and further optimized using ABC’s algebraic rewriting and technology-independent optimization passes [4, 12].

2.1.1 Synthesis with Constraints. These three inputs are synthesized into an And-Inverter Graph (AIG) representation with CRISP constraints embedded as assumptions using the open-source synthesizer Synlig [6]. The RTL is first elaborated and the microarchitecture configuration specifies where constraint assertions should be injected into the design hierarchy. The resulting AIG encodes both the circuit structure and the behavioral constraints under which optimization must preserve correctness.

2.1.2 Microarchitecture-Aware Error Injection. Rather than exhaustively injecting errors at every signal in the design, PDAG-SFA leverages the microarchitecture configuration and CRISP constraints to identify promising optimization candidates. By analyzing which instructions are forbidden and examining the cores’ multiplexers’ structures, the tool determines which functional unit outputs are likely unobservable. For example, if CRISP constraints forbid all division instructions, the microarchitecture configuration identifies the divider output in the ALU result mux, enabling targeted error injection at this signal. This microarchitecture-aware targeting dramatically reduces the verification problem space compared to naïve enumeration of all possible error injection sites, while maintaining soundness.

For each error injection candidate, PDAG-SFA must prove that the injected error cannot propagate to architecturally visible state (register file, CSRs, memory interface). Rather than expensive symbolic simulation or unbounded LTL model checking, PDAG-SFA employs bounded Sequential Equivalence Checking (SEC) using k -induction. The microarchitecture configuration guides this verification in two critical ways. First, the pipeline depth parameter sets the induction depth k , reflecting the insight that signals typically stabilize within one pipeline traversal, providing a natural bound for inductive proofs. Second, only architecturally visible state identified in the configuration (register file, CSRs, flags) serves as Primary Outputs (POs) for equivalence checking, avoiding unnecessary verification of internal microarchitectural state. The iterative refinement loop (Fig. 2) employs BMC refinement and speculative reduction to efficiently construct and solve the bounded SEC problems, translating them to satisfiability queries on time-unrolled circuits. This reduces computational complexity from EXPTIME (for unbounded LTL model checking) to coNP (for bounded SAT).

2.1.3 Sequential SAT Sweeping. After error injection-based optimization, PDAG-SFA applies sequential SAT sweeping [12] as a complementary, orthogonal optimization technique. Sequential SAT sweeping identifies functionally equivalent signals within the circuit by first simulating with random inputs and hashing signal value sequences to find potentially equivalent net pairs. Candidate equivalences are then proven or disproven via k -induction: both the base case (equivalence for the first $k - 1$ cycles) and the inductive step (equivalence in cycle k follows from equivalence in the previous $k - 1$ cycles) reduce to bounded SEC problems solved via SAT. Critically, these equivalence checks are performed under the CRISP constraints embedded as assumptions in the AIG. The SAT solver only needs to consider behaviors permitted by the ISA-level, data-level, and interface-level constraints, enabling proofs of equivalence that would be invalid in the unconstrained circuit. This technique— the combination of microarchitecture-aware error injection and constraint-aware sequential SAT sweeping yields a three orders of magnitude improvement in runtime compared to unbounded verification approaches (Table 2).

2.1.4 Optimization and Output. Proven equivalences are merged into the AIG, eliminating signals shown to be unobservable under the given constraints. The optimized AIG then undergoes further technology-independent optimization using ABC’s algebraic rewriting (rewrite), functional reduction via AIGs (fraig), and

structural balancing (balance) passes [4]. The final optimized AIG can be mapped to standard cells for physical implementation.

Table 1: Core area (Skywater 130 nm) of a single-cycle RV32E microprocessor under various ISA subset assumptions. A ✓ indicates the presence of instructions using the hardware multiplier, divider, adder, and barrel shifter, while a ✗ indicates the absence of those instructions. Cores generated by PDAG-SFA are, without exception, smaller than those generated through RISSP’s additive processing.

Mul	Div	Add	Shift	RISSP (μm ²)	PDAG-SFA (μm ²)	%Δ
✗	✗	✗	✗	33148.04	32205.90	-2.84%
✗	✗	✗	✓	36862.85	35661.70	-3.26%
✗	✗	✓	✗	35940.72	32205.90	-10.39%
✗	✗	✓	✓	39303.95	35609.20	-9.40%
✗	✓	✗	✗	60857.12	31996.90	-47.42%
✗	✓	✗	✓	64291.66	35521.60	-44.75%
✗	✓	✓	✗	63397.05	31950.60	-49.60%
✗	✓	✓	✓	66939.20	35376.40	-47.15%
✓	✗	✗	✗	85254.27	77455.50	-9.15%
✓	✗	✗	✓	82890.75	81249.20	-1.98%
✓	✗	✓	✗	85784.77	77583.20	-9.56%
✓	✗	✓	✓	88697.57	80835.00	-8.86%
✓	✓	✗	✗	81197.88	77586.90	-4.45%
✓	✓	✗	✓	83980.54	80808.80	-3.78%
✓	✓	✓	✗	100503.89	77327.90	-23.06%
✓	✓	✓	✓	104164.90	80835.00	-22.40%

Table 1 demonstrates that PDAG-SFA’s gate-level analysis consistently captures equal or greater area reductions compared to RISSP’s module-level additive approach across all ISA subset configurations. While both methods remove entire functional units when their instructions are forbidden (e.g., removing the divider when division instructions are unused), PDAG-SFA achieves 44–50% greater reduction in these cases by identifying additional optimization opportunities in control logic, multiplexing networks, and glue logic surrounding the removed modules. More significantly, even when all functional units are required (final row: all modules present), PDAG-SFA still achieves 22% area reduction—a capability entirely beyond RISSP’s reach. This phenomenon, first identified in the original Property-Driven Analysis and Generation (PDAG) work [3], arises because gate-level analysis exploits ODCs within functional units themselves. Even when a module is necessary, ISA constraints render portions of its internal logic unobservable: a multiplier may contain special-case handling for zero operands, sign extension logic for narrower data types, or overflow detection circuits that become redundant under specific instruction subset constraints. RISSP’s module-level approach must include or exclude entire functional units as monolithic blocks, while PDAG-SFA’s AIG-level signal correspondence analysis systematically identifies and eliminates unobservable gates within required modules.

We compared PDAG-SFA against FSYN [9] using a GenMul-generated 32×32 multiplier with randomly generated external constraints (4 input bits fixed to zero), following [9]. Table 3 compares PDAG-SFA against both FSYN variants: without filtering and with the optional rule filtering phase (40 stimuli). On this benchmark, PDAG-SFA achieves 26.69% gate reduction versus FSYN’s 13.88% while completing in 7 seconds versus 2.5 hours. This suggests that

Table 2: Runtime for PDAG-SFA and PDAG on 2-stage version of Ibex [14]. PDAG analysis was performed with JasperGold with timeout set for five core hours.

ISA Subset	PDAG-SFA	Completed	PDAG	Completed
RV32IM	12 s	✓	> 5 h	✗
RV32I	12 s	✓	> 5 h	✗

semiformal signal correspondence can be both significantly faster and more effective than formal verification for constraint-based optimization.

We attempted additional evaluation on EPFL benchmarks [1], including mem_ctrl and arbiter; however, the constraint configurations used in [9] for these circuits were not specified. Our trials with randomly generated constraints resulted in runtimes exceeding 5 hours or zero optimization, indicating that further investigation of appropriate constraint selection for these designs is needed.

Table 3: Gate Reduction Comparison: PDAG-SFA vs FSYN on GenMul 32×32 Multiplier

Metric	FSYN (no filter)	FSYN (filter-40)	PDAG-SFA
Initial size (gates)	11,563	11,563	11,967
Optimized size (gates)	9,958	9,986	8,773
Gates removed	1,605	1,577	3,194
Reduction (%)	13.88	13.64	26.69
Runtime (h:mm:ss)	2:28:57	2:15:55	0:00:07

3 CRISP

In order to effectively express application specific constraints and automatically generate corresponding SystemVerilog assertion, we use a domain-specific language to imperatively add and remove ISA subsets from an (initially empty) allowed subset. Prior works force users to express constraints by manually construct SVA, which can be both tedious and buggy.

To support this, the DSL supports two statements:

Let I be the set of all RISC-V instructions. The ‘state’ of the language at any point of processing is a set of allowed instructions, which we denote $S_{current} \subseteq I$. The semantic domain for a CRISP program (or, sequence of statements) is a function that maps an initial set S_0 to a final state, S_f .

Let $\mathcal{E}$ be the semantic function mapping expressions in CRISP to RISC-V instructions, i.e., $\mathcal{E} : \text{Expr} \rightarrow \mathcal{P}(I)$. Let $\mathcal{S}$ be the semantic function mapping statements to a state transformer function, i.e., $\mathcal{S} : \text{Stmt} \rightarrow (\mathcal{P}(I) \rightarrow \mathcal{P}(I))$. Let $\mathcal{P}$ be the semantic function mapping programs to a state-transition function: $\mathcal{P} : P \rightarrow (\mathcal{P}(I) \rightarrow \mathcal{P}(I))$. The meaning of program p is given by $(\mathcal{P}(p))(\emptyset)$.

3.1 Expression Syntax and Semantics

CRISP expressions denote sets of RISC-V instructions with five forms:

$$E ::= \text{ext} \mid \text{instr} \mid \text{instr}\{C\} \mid r_1-r_2$$

where extensions (e.g., RV32I) expand to instruction sets, individual instructions (e.g., ADD) to singletons, and constrained instructions apply field restrictions. Field constraints $C = \{f = v \mid f \text{ in } r_1\text{-}r_2\}$ specify equality or range restrictions for fields $f \in \{\text{rd}, \text{rs1}, \text{rs2}, \text{imm}\}$.

The semantic function $\mathcal{E}$ maps expressions to instruction sets where each instruction $\iota \in I$ is encoded as (name, pattern, mask, fields):

$$\mathcal{E}[[\text{ext}]] = \{\iota \in I \mid \iota.\text{name} \in \text{instructions}(\text{ext})\} \quad (1)$$

$$\mathcal{E}[[\text{instr}\{C\}]] = \{\text{constrain}(\iota, C) \mid \iota \in \mathcal{E}[[\text{instr}]]\} \quad (2)$$

The constraint function refines encodings by updating pattern/mask bits: fixing bits for equality constraints, encoding common prefixes for ranges.

3.2 Statement Syntax and Semantics

CRISP statements transform instruction sets through two operations:

$$S ::= \text{include } E \mid \text{forbid } E \quad (3)$$

with semantics:

$$\mathcal{S}[[\text{include } E]] = \lambda s . s \cup \mathcal{E}[[E]] \quad (4)$$

$$\mathcal{S}[[\text{forbid } E]] = \lambda s . s \setminus \mathcal{E}[[E]] \quad (5)$$

Programs are statement sequences with semantics defined by function composition:

$$\mathcal{P}[[\text{ }]] = \lambda s . s \quad (6)$$

$$\mathcal{P}[[\text{Stmt}_1; P_{\text{rest}}]] = \mathcal{P}[[P_{\text{rest}}]] \circ \mathcal{S}[[\text{Stmt}_1]] \quad (7)$$

The meaning of program p is $\mathcal{P}[[p]](\emptyset)$, processing statements left-to-right from an empty set. For example, `include RV32I, forbid x16-x31` first adds all RV32I instructions, then removes those using registers x16-x31.

3.3 Implementation

CRISP is implemented in Python, representing simple constraints via ranges or enumerations on instruction fields, and complex cross-field constraints via AIGs. The final expression converts to SystemVerilog assumptions injected into RTL/netlists.

3.4 μ Arch Config File

The configuration YAML serves three critical functions: (1) specifies microarchitectural signals (clock, reset, instruction decoder) and injection points for SVA assertions, (2) defines pipeline depth for k-induction tuning and result multiplexer structure for observability analysis, and (3) enables non-ISA constraints including data-width restrictions (e.g., limiting operations to 8/16-bit data) and timing-level constraints (e.g., fixed cache latencies) that expose optimization opportunities invisible to ISA-only approaches. A universal template is provided with minor per-core customization required.

4 EXPERIMENTAL RESULTS

We evaluate PDAG-SFA on the Ibex RISC-V core, an open-source 32-bit microcontroller implementing the RV32IMC instruction set. Our

experiments demonstrate the framework’s ability to optimize hardware under three distinct constraint categories: architectural register constraints, ISA subset constraints, and data-width constraints. All synthesis results target the SkyWater 130 nm open-source PDK using standard optimization settings.

Experimental Setup. We apply PDAG-SFA to generate specialized Ibex variants under different application-specific constraints expressed in our CRISP. Each configuration undergoes the complete optimization flow: constraint specification, semiformal verification to identify unused hardware, and automated removal of unneeded logic. The resulting netlists undergo technology mapping using ABC and static timing analysis with OpenSTA to evaluate area reduction and timing impact. We measure two key metrics: core area and maximum operating frequency (FMax).

Register File Optimization. Figure 3(a) presents the impact of constraining the number of architectural registers from the standard 32 down to 4 registers. This optimization targets embedded applications with limited register pressure, such as simple control loops and state machines. The results demonstrate near-linear area scaling: reducing to 4 registers achieves a 47.3 % area reduction (from 89 873.7 μm^2 to 47 314.1 μm^2) while improving FMax by 80.1 % (from 26.90 MHz to 48.45 MHz). The frequency improvement arises from reduced multiplexer complexity and shorter critical paths in the register file decode logic.

ISA Subset Optimization. Figure 3(b) illustrates the area and performance impact of removing different instruction categories from the base ISA. Starting from the full RV32IM configuration (126 176.0 μm^2), we evaluate progressively restricted instruction sets. Removing the multiply/divide instructions (RV32I) reduces area by 28.7 % to 89 873.7 μm^2 while improving FMax from 22.0 MHz to 26.9 MHz. Further restrictions yield diminishing returns: removing shift instructions saves only an additional 2.7 % area, while an arithmetic-only core (supporting only ADD, SUB, and basic ALU operations) achieves 34.7 % total area reduction compared to RV32IM. Notably, removing branches has minimal impact (0.2 % area reduction), as branch logic shares substantial resources with the base control flow infrastructure that must remain.

Data-Width Optimization. Figure 3(c) demonstrates PDAG-SFA’s unique ability to optimize based on data-level constraints that cannot be expressed as ISA restrictions. While RV32IM natively supports only 32-bit operations, many embedded applications operate on smaller data types. By analyzing the actual data widths used in target applications, we can eliminate unused portions of the datapath. Constraining operations to 8-bit data reduces area by 47.6 % (from 89 851.2 μm^2 to 47 125.2 μm^2) and improves FMax by 63.5 %. The 16-bit configuration achieves a balanced trade-off with 31.0 % area reduction while maintaining reasonable performance. These optimizations are particularly valuable for sensor processing and control applications where data naturally fits in smaller widths.

Timing Level Constraint. Table 4 demonstrates the impact of timing-level constraints on the Ibex core area. Ibex employs a request-grant-valid protocol for its data cache (dcache) and instruction cache (icache) interfaces. By constraining the behavior of

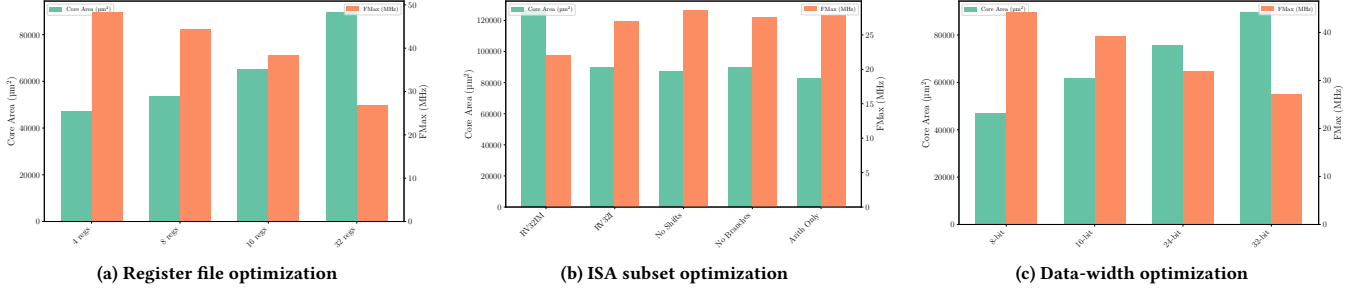


Figure 3: Ibex core optimization results under different constraint categories. (a) Constraining architectural registers from 32 to 4 achieves 47.3% area reduction. (b) ISA subsetting from RV32IM to arithmetic-only achieves 34.7% area reduction. (c) Data-width constraints (impossible to express as ISA constraints) achieve 47.6% area reduction for 8-bit operations. All configurations show consistent FMax improvements with area reduction.

dcache/icache accesses, the hardware required for protocol implementation can be simplified. Specifically, we constrain the cache access latency such that hits complete in 1 cycle, while misses are guaranteed to resolve within 4 cycles and 5 cycles for dcache and icache, respectively. With timing-level constraints, PDAG-SFA achieves an additional 0.19% area reduction. The improvement is minimal due to the simplicity of Ibex’s memory subsystem. However, this demonstrates the capability of applying system-specific constraints at the interface level. For microarchitectures with complex memory systems, timing-level constraints may yield more substantial area reductions. Additionally, such constraints can be used to help resolve timing side channel [10]. For example, it can help expand the set of instructions suitable for fixed-latency programming, effectively enlarging the ‘safe’ ISA subset [8]. Note that it is the user’s responsibility to determine which cache behaviors are appropriate for their specific system requirements.

Table 4: Area Comparison Showing Impact of Timing Level Constraint

Configuration	Area (µm ²)		Area Reduction (%)
	Without Timing Level Constraint	With Timing Level Constraint	
RV32IM	126,860.42	126,622.69	0.19
RV32IM_no_mul	99,206.40	99,075.02	0.13
RV32I	89,851.17	89,706.04	0.16

Discussion. The experimental results validate PDAG-SFA’s effectiveness across diverse optimization scenarios. The framework successfully identifies and removes unused hardware at multiple abstraction levels, from architectural registers to individual datapath bits. The consistent FMax improvements accompanying area reductions demonstrate that removing unused logic not only saves silicon area but also simplifies critical paths. Most significantly, the data-width optimizations showcase PDAG-SFA’s microarchitecture-aware capabilities, achieving substantial savings impossible with ISA-only approaches. These results, obtained in 2 to 4 orders of magnitude less time than exhaustive formal verification, confirm

that semiformal analysis provides a practical path to aggressive hardware specialization without sacrificing correctness guarantees for the constrained usage scenarios.

5 CONCLUSION

This work presented PDAG-SFA, a scalable hardware pruning framework that addresses the fundamental challenges limiting practical ASIP deployment. By embracing semiformal verification techniques and microarchitecture awareness, we overcome the computational intractability and optimization limitations of prior approaches. Our key contributions include: (1) a semiformal verification methodology that trades completeness for practical scalability, enabling 2 to 4 orders of magnitude faster analysis than exhaustive formal methods; (2) a microarchitecture-aware optimization approach that translates diverse constraints including ISA-level, data-level, and interface-level specifications into exploitable don’t-care conditions; and (3) the CRISP, a domain-specific language with formal denotational semantics for precise constraint specification.

Experimental evaluation on the Ibex RISC-V core demonstrates the framework’s effectiveness across multiple optimization dimensions. Register file optimizations achieve up to 47.3% area reduction with 80.1% FMax improvement. ISA subsetting yields 34.7% area savings for arithmetic-only cores. Most significantly, data-width constraints that are impossible to express in ISA-only frameworks enable 47.6% area reduction for 8-bit applications. These results, synthesized using the SkyWater 130 nm open-source PDK with ABC technology mapping and OpenSTA timing analysis, validate our approach’s practical applicability.

The implications extend beyond immediate area and performance gains. PDAG-SFA’s open-source implementation democratizes advanced ASIP design, particularly crucial for emerging domains like flexible and printed electronics where EDA licensing costs can exceed fabrication expenses. The framework’s compatibility with arbitrary microarchitectures, unlike template-based approaches, preserves design flexibility while enabling aggressive optimization. As embedded systems proliferate and transistor budgets tighten, PDAG-SFA provides a practical path toward ultra-efficient, application-specific processors that extract maximum value from every transistor.

REFERENCES

- [1] Luca Amarú, Pierre-Emmanuel Gaillardon, and Giovanni De Micheli. 2015. The EPFL combinational benchmark suite. *Hypotenuse* 256, 128 (2015), 214335.
- [2] Nathaniel Bleier, Calvin Lee, Francisco Rodriguez, Antony Sou, Scott White, and Rakesh Kumar. 2022. FlexiCores: low footprint, high yield, field reprogrammable flexible microprocessors. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) (ISCA '22). Association for Computing Machinery, New York, NY, USA, 831–846. <https://doi.org/10.1145/3470496.3527410>
- [3] Nathan Bleier, John Sartori, and Rakesh Kumar. 2021. Property-driven Automatic Generation of Reduced-ISA Hardware. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. 349–354. <https://doi.org/10.1109/DAC18074.2021.9586090>
- [4] Robert Brayton and Alan Mishchenko. 2010. ABC: An academic industrial-strength verification tool. In *International Conference on Computer Aided Verification*. Springer, 24–40.
- [5] Hari Cherupalli, Henry Duwe, Weidong Ye, Rakesh Kumar, and John Sartori. 2017. Bespoke Processors for Applications with Ultra-low Area and Power Constraints. In *Proceedings of the 44th Annual International Symposium on Computer Architecture* (Toronto, ON, Canada) (ISCA '17). Association for Computing Machinery, New York, NY, USA, 41–54. <https://doi.org/10.1145/3079856.3080247>
- [6] CHIPS Alliance. 2024. Synlig: SystemVerilog Synthesis Tool. <https://github.com/chipsalliance/synlig>.
- [7] Krishanu Debnath, Rajeev Murgai, Mayank Jain, and Janet Olson. 2018. SAT-based redundancy removal. In *2018 Design, Automation Test in Europe Conference Exhibition (DATE)*. 315–318. <https://doi.org/10.23919/DATE.2018.8342028>
- [8] Sushant Dinesh, Yongye Zhu, and Christopher W. Fletcher. 2025. H-Houdini: Scalable Invariant Learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 603–618. <https://doi.org/10.1145/3669940.3707263>
- [9] Daniel Große, Lucas Klemmer, and Dominik Bonora. 2024. Using Formal Verification Methods for Optimization of Circuits Under External Constraints. In *2024 Design, Automation Test in Europe Conference Exhibition (DATE)*. 1–6. <https://doi.org/10.23919/DATE58400.2024.10546556>
- [10] Nate Lawson. 2009. Side-channel attacks on cryptographic software. *IEEE Security & Privacy* 7, 6 (2009), 65–68.
- [11] Gabriel Cadilha Marques, Farhan Rasheed, Jasmin Aghassi-Hagmann, and Mehdi B. Tahoori. 2018. From silicon to printed electronics: A coherent modeling and design flow approach based on printed electrolyte gated FETs. In *2018 23rd Asia and South Pacific Design Automation Conference (ASP-DAC)*. 658–663. <https://doi.org/10.1109/ASPDAC.2018.8297397>
- [12] Alan Mishchenko, Michael Case, Robert Brayton, and Stephen Jang. 2008. Scalable and scalably-verifiable sequential synthesis. In *2008 IEEE/ACM International Conference on Computer-Aided Design*. 234–241. <https://doi.org/10.1109/ICCAD.2008.4681580>
- [13] Alireza Raisiardi, Konstantinos Iordanou, Jędrzej Kufel, Kowshik Gudimetla, Kris Myny, and Emre Ozer. 2025. Flexing RISC-V Instruction Subset Processors to Extreme Edge. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*. Association for Computing Machinery, New York, NY, USA, 1147–1159. <https://doi.org/10.1145/3725843.3756036>
- [14] Pasquale Davide Schiavone, Francesco Conti, Davide Rossi, Michael Gautschi, Antonio Pullini, Eric Flamand, and Luca Benini. 2017. Slow and steady wins the race? A comparison of ultra-low-power RISC-V cores for Internet-of-Things applications. In *2017 27th International Symposium on Power and Timing Modeling, Optimization and Simulation (PATMOS)*. IEEE, 1–8.
- [15] Subhash Sethumurugan, Shashank Hegde, Hari Cherupalli, and John Sartori. 2022. A scalable symbolic simulation tool for low power embedded systems. In *Proceedings of the 59th ACM/IEEE Design Automation Conference* (San Francisco, California) (DAC '22). Association for Computing Machinery, New York, NY, USA, 175–180. <https://doi.org/10.1145/3489517.3530433>
- [16] Qilin Si, Prattay Chowdhury, Rohit Sreekumar, and Benjamin Carrion Schafer. 2023. Application Specific Approximate Behavioral Processor. *IEEE Transactions on Sustainable Computing* 8, 2 (2023), 165–179. <https://doi.org/10.1109/TSUSC.2022.3222117>
- [17] Stian Gerlach Sørensen, Christian Bartsch, Dominik Stoffel, and Wolfgang Kunz. 2022. Generation of Formal CPU Profiles for Embedded Systems. In *2022 IFIP/IEEE 30th International Conference on Very Large Scale Integration (VLSI-SoC)*. 1–6. <https://doi.org/10.1109/VLSI-SoC54400.2022.9939572>
- [18] Qinhan Tan, Aarti Gupta, and Sharad Malik. 2022. Usage-Based RTL Subsetting for Hardware Accelerators. In *2022 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9.